This document contains information regarding the contents of the source codes that were the basis for our research on cybersecurity of smart contracts (currently under review at ACM CCS 2026). We introduce general architecture of the implemented DEX application in Section 1, with emphasis on the smart contracts (Section 1.2). All injected vulnerabilites are described in details in Section 2, along with listing of both vulnerable and secure source codes per each of the implemented SWC vulnerabilities (in Sections 2.1-2.29).

1 Decentralized Exchange - general architecture

To support the objectives of this study, a simple decentralized exchange (DEX) application was developed. The purpose of this application is to serve as an experimental environment for evaluating the security of smart contracts and the effectiveness of vulnerability detection tools. The DEX was implemented as a Solidity smart contract and a simple front-end. The main view in the application is shown on the figure 1.

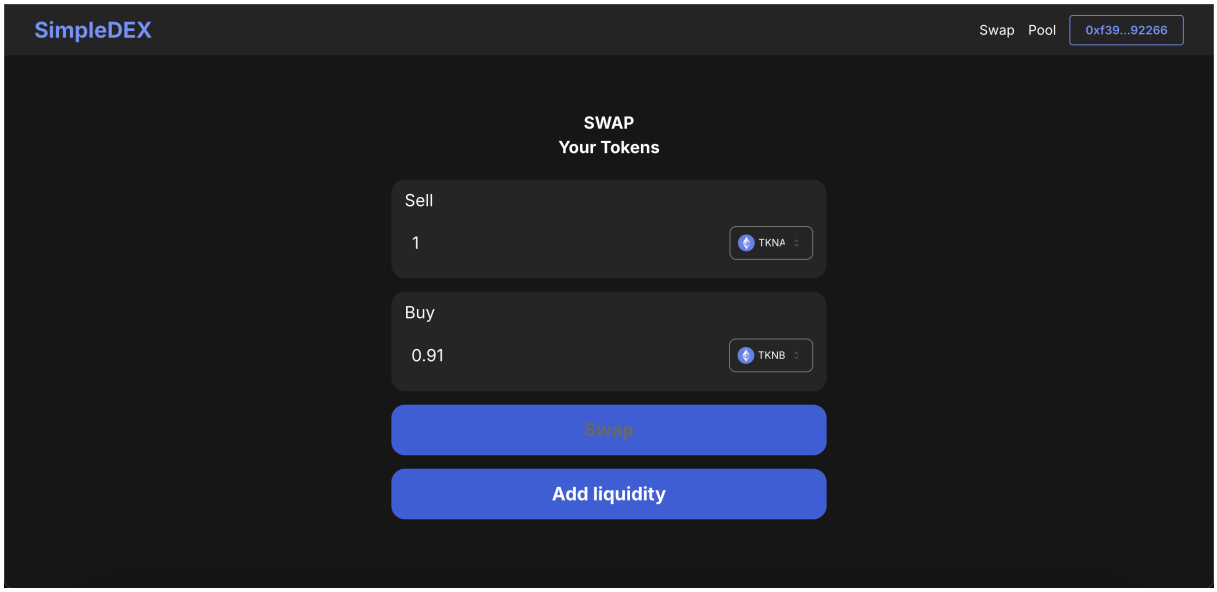

Figure 1: Figure showing the main view of the application.

The exchange allows users to perform basic operations typical of decentralized trading platforms. Its core functionalities include:

- depositing assets into the smart contract,
- withdrawing assets from the smart contract,
- swapping between assets.

To enable meaningful analysis, several common vulnerabilities were intentionally introduced into the smart contract. These vulnerabilities are based on the SWC Registry

and represent realistic security issues that occur in real-world applications. The DEX does not aim to provide production-level reliability or safety but rather to offer a controlled environment for testing and comparison of automated vulnerability detection tools.

1.1 DEX architecture

Application can be divided into two parts depending on placing its components:

- on-chain part — data stored on blockchain network,
- off-chain part — data stored outside blockchain network.

In off-chain part there is front-end of the application, that communicates with web browser. It is layer of the application that is displayed to users and users can directly interact with it. Front-end can both receive and send data to web browser. Between off-chain and on-chain parts there are providers - services that allow application to communicate with blockchain network. They can receive and send data to off-chain and on-chain parts. Only one provider is part of the application and it is digital cryptocurrency wallet. The on-chain part is smart contract that communicates with blockchain network via Ethereum Virtual Machine (EVM). The main task of this smart contract is to exchange users' assets on blockchain network, providing and removing liquidity of the smart contract. The architecture was shown on the figure 2.

1.2 Smart contract implementation

The smart contract within the overall application is responsible for the ability to exchange various assets with each other. These assets are ERC20 tokens. Users can provide the smart contract with liquidity in the form of ERC20 tokens (the `addLiquidity()` function). Users can then take benefit of the liquidity pool by exchanging one token for another (the `swap()` function). After providing liquidity to the smart contract, users can also withdraw the previously provided funds (the `removeLiquidity()` function). There are a lot functions that have been implemented to highlight certain vulnerabilities of the smart contract, for example `loopUsers()` function. The smart contract was implemented using Solidity language.

The smart contract was implemented in Solidity compilation version 0.4.24, but Securify tool does not analyze smart contracts in version below 0.5.8. Due to this fact, alternative version of this smart contract was implemented using compiler version 0.5.8. Both versions were tested using all selected tools.

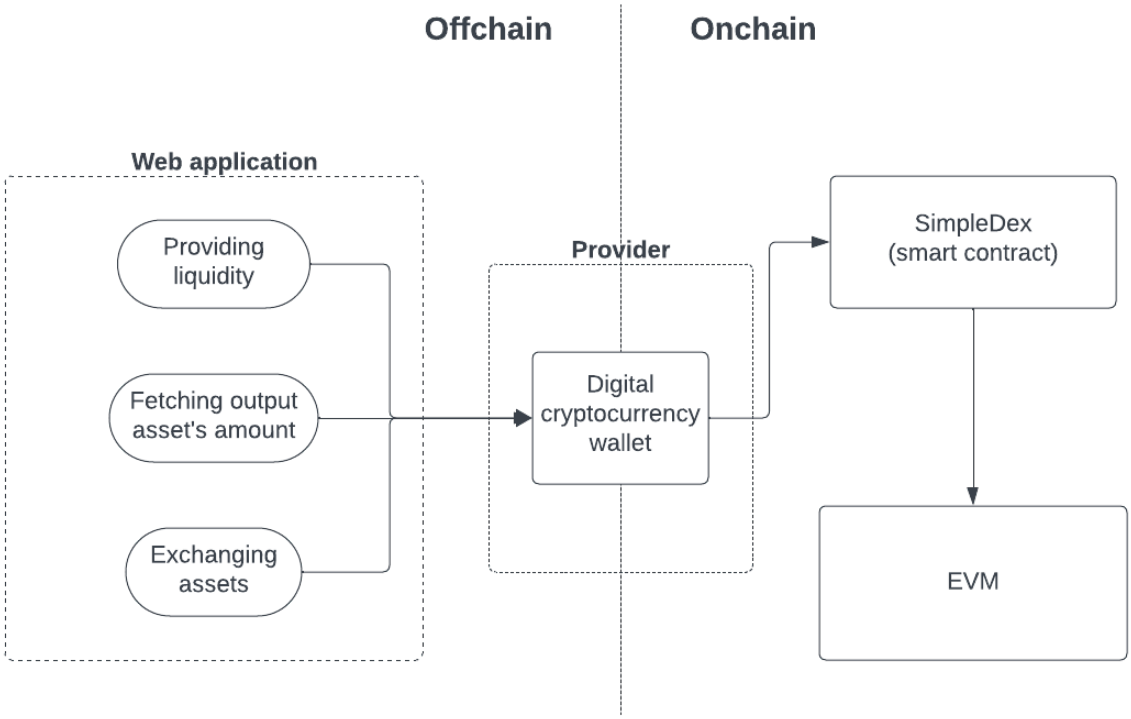

Figure 2: Figure showing architecture of the application.

2 Injected vulnerabilities

A smart contract was implemented for the experiments, in which certain vulnerabilities were intentionally introduced. The SWC Registry defines a total of 37 vulnerabilities. Out of these, 29 were intentionally introduced into the smart contract and will be presented in sections 3.4.1 - 3.4.29. The remaining 8 vulnerabilities could not be incorporated due to the specific nature and limitations of the developed smart contract. These vulnerabilities are:

- Unprotected Ether Withdrawal (SWC-105)

  This vulnerability has not been implemented, because the smart contract does not contain any function that allows the withdrawal of accumulated Ether, making it impossible to present a lack of control over such an operation.

- Transaction Order Dependence (SWC-114)

  The SWC-114 vulnerability was excluded, as the smart contract does not contain any functions whose core logic or calculations depend on global variables that are modified within the same transaction - which is necessary to take advantage of transaction order.

- Requirement Violation (SWC-123)

The key assumptions of the functions (e.g. correct `require()` conditions) were intentionally left intact to avoid interfering with the testing of other vulnerabilities within the same functions.

- Write to Arbitrary Storage Location (SWC-124)

  This vulnerability requires the use of low-level assembly operations, which is complex and difficult to replicate in a simple decentralized exchange application in a controlled manner.

- Incorrect Inheritance Order (SWC-125)

  The SimpleDex smart contract does not use inheritance, so it was not possible to present a wrong inheritance order of base classes.

- Insufficient Gas Griefing (SWC-126)

  To effectively demonstrate this vulnerability, it would be necessary to include an external smart contract or recipient with unknown behavior (e.g., with a high execution cost), which is difficult to execute in the current version of the smart contract due to its relative simplicity.

- Arbitrary Jump with Function Type Variable (SWC-127)

  This vulnerability requires using function pointers and manipulating their value in an uncontrolled manner, which has not been implemented due to its unsuitability in the context of the implemented application.

- Right-To-Left-Override control character (U+202E) (SWC-130)

  This vulnerability is based on the use of invisible control characters in the smart contract file or variable names to mislead, which is difficult to demonstrate in the source code of the smart contract itself without affecting its readability or compilation.

The rest of the vulnerabilities classified in the SWC Registry have been introduced into the smart contract of the DEX application. Below is a description of each vulnerability, together with its implementation in the smart contract and an example proposal for remediation. Despite the weaknesses introduced, the smart contract compiles successfully and remains functional within the context of the sample application, all defined operations can still be executed. However, the smart contract is now prone to attacks based on the vulnerabilities described below.

2.1 Function Default Visibility (SWC-100)

In Solidity, the lack of explicitly declared visibility of a function results in it being given the default visibility of public. This can lead to inadvertent access to internal functions,

increasing the risk of unauthorized access or use of the smart contract in a way not anticipated by its creator.

The way the vulnerability was introduced into the source code is shown in the listing 1.

```
1 function removeLiquidity(address tokenAddress, uint share) {
2     uint amount = share * reserves[tokenAddress];
3
4     IERC20(tokenAddress).transfer(msg.sender, amount);
5
6     shares[msg.sender] -= share;
7     reserves[tokenAddress] -= amount;
8 }
```

Listing 1: Implementation of the SWC-100: Function Default Visibility vulnerability in the smart contract

In the listing 1, line 1, the function `removeLiquidity` is defined without a visibility specifier. In Solidity versions 0.4.x, the default visibility is public, which means anyone can call this function. This creates the SWC-100: Function Default Visibility vulnerability.

An attacker could exploit this by calling `removeLiquidity` function directly and withdrawing tokens from the reserves, leading to loss of funds.

The way the vulnerability could be remediated is shown in the listing 2.

```
function removeLiquidity(address tokenAddress, uint share)
    external {
     uint amount = share * reserves[tokenAddress];

     IERC20(tokenAddress).transfer(msg.sender, amount);

     shares[msg.sender] -= share;
     reserves[tokenAddress] -= amount;
}
```

Listing 2: Implementation of the remediation to the injected the SWC-100: Function Default Visibility vulnerability

In the listing 2, line 1, the function is explicitly marked as external. This makes the visibility clear and prevents the unintended default public access. As a result, the vulnerability is fixed.

2.2 Integer Overflow and Underflow (SWC-101)

This vulnerability occurs when arithmetic operations on integers exceed their maximum or minimum value. In older versions of Solidity (before version 0.8.0), there was no automatic protection - the counter would “wrap around”, which could lead to unpredictable results and security vulnerabilities.

The way the vulnerability was introduced into the source code is shown in the listing 3.

```
1 function swap(address tokenIn, uint amountIn, address tokenOut)
    external {
2     IERC20(tokenIn).transferFrom(msg.sender, address(this),
        amountIn);
3
4     (uint reserveIn, uint reserveOut) = (reserves[tokenIn],
        reserves[tokenOut]);
5
6     uint amountOut = (amountIn * reserveOut) / (reserveIn +
        amountIn);
7
8     IERC20(tokenOut).transfer(msg.sender, amountOut);
9
10    reserves[tokenIn] += amountIn;
11    reserves[tokenOut] -= amountOut;
12 }
```

Listing 3: Implementation of the SWC-101: Integer Overflow and Underflow vulnerability in the smart contract

In the listing 3, the function `swap` performs arithmetic operations directly on unsigned integers. This can cause SWC-101: Integer Overflow and Underflow. For example, if `reserves[tokenIn]` value is very large, the addition in line 10 could overflow, leading to incorrect results and potential loss of funds.

The way the vulnerability could be remediated is shown in the listing 4.

```
import "@openzeppelin/contracts/math/SafeMath.sol";

using SafeMath for uint256;

function swap(address tokenIn, uint amountIn, address tokenOut)
    external {
    IERC20(tokenIn).transferFrom(msg.sender, address(this),
        amountIn);

    (uint reserveIn, uint reserveOut) = (reserves[tokenIn],
        reserves[tokenOut]);

    uint amountOut = (amountIn * reserveOut) / (reserveIn +
        amountIn);
```

```
11
12      IERC20(tokenOut).transfer(msg.sender, amountOut);
13
14      reserves[tokenIn] = reserves[tokenIn].add(amountIn);
15      reserves[tokenOut] = reserves[tokenOut].sub(amountOut);
16  }
```

Listing 4: Implementation of the remediation to the injected the SWC-101: Integer Overflow and Underflow vulnerability

In the listing 4, the smart contract uses OpenZeppelin SafeMath. In lines 14 and 15, the operations are replaced with `add` and `sub` functions, which automatically check for overflows and underflows. If such a case happens, the transaction reverts instead of producing a wrong value. This removes the SWC-101 vulnerability.

2.3 Outdated Compiler Version (SWC-102)

Using outdated versions of the Solidity compiler may expose the smart contract to known security vulnerabilities that have already been fixed in newer versions. It is recommended to always use an up to date, stable version of the compiler and to accurately declare the version in the `pragma` directive. For example, newer versions of Solidity introduced:

- Automatic overflow and underflow checks (SWC-101) – starting from 0.8.0, arithmetic operations revert on overflow, preventing errors that previously could be exploited.
- More predictable error handling (SWC-110) – changes in how assert, require, and revert work reduced the risk of unexpected gas consumption and denial-of-service scenarios.
- Better protections against reentrancy (SWC-107) – compiler improvements, like clearer checks for function visibility and stricter rules for call.value(), made such attacks easier to avoid.

The way the vulnerability was introduced into the source code is shown in the listing 5.

```
1  pragma solidity ^0.5.8;
```

Listing 5: Implementation of the SWC-102: Outdated Compiler Version vulnerability in the smart contract

The way the vulnerability could be remediated is shown in the listing 6.

```
pragma solidity ^0.8.30;
```

Listing 6: Implementation of the remediation to the injected the SWC-102: Outdated Compiler Version vulnerability

2.4 Floating Pragma (SWC-103)

Declaring a range of compiler versions (e.g., ^0.8.0) instead of one specific version can lead to unpredictable behavior if the smart contract is compiled using a newer, untested compiler. This practice makes auditing more difficult and can introduce new vulnerabilities. For example, using a floating pragma may unintentionally expose a smart contract to:

- SWC-101 (Integer Overflow and Underflow) - in compiler versions before 0.8.0 arithmetic operations did not have built-in overflow checks, while from 0.8.x such checks are automatic. A smart contract tested on Solidity 0.8.x could silently become unsafe if compiled with an older version without these protections.
- SWC-110 (Assert Violation) – the behavior of `assert()` changed between compiler versions. In some versions it consumed all gas, in others it reverted and refunded unused gas. This can change how DoS risks appear in smart contracts.

The way the vulnerability was introduced into the source code is shown in the listing 7.

```
pragma solidity ^0.5.8;
```

Listing 7: Implementation of the SWC-103: Floating Pragma vulnerability in the smart contract

The way the vulnerability could be remediated is shown in the listing 8.

```
pragma solidity 0.5.8;
```

Listing 8: Implementation of the remediation to the injected the SWC-103: Floating Pragma vulnerability

2.5 Unchecked Call Return Value (SWC-104)

The vulnerability occurs when a smart contract makes a low-level call (e.g. `call`, `send`, `delegatecall`) but does not check whether it was successful. This can lead to incorrect states or loss of resources if the operation fails and the smart contract continues to act as if everything went correctly.

The way the vulnerability was introduced into the source code is shown in the listing 9.

```
function swap(address tokenIn, uint amountIn, address tokenOut) external {
    IERC20(tokenIn).transferFrom(msg.sender, address(this), amountIn);

    (uint reserveIn, uint reserveOut) = (reserves[tokenIn], reserves[tokenOut]);

    uint amountOut = (amountIn * reserveOut) / (reserveIn + amountIn);

    IERC20(tokenOut).transfer(msg.sender, amountOut);

    reserves[tokenIn] += amountIn;
    reserves[tokenOut] -= amountOut;
}
```

Listing 9: Implementation of the SWC-104: Unchecked Call Return Value vulnerability in the smart contract

In the listing 9, the function `swap` calls `transferFrom` and `transfer` functinos without checking their return values. This creates the SWC-104: Unchecked Call Return Value vulnerability. If one of these calls fails but does not revert, the function would continue execution as if the transfer succeeded. This could let tokens be stolen or reserves become inconsistent.

The way the vulnerability could be remediated is shown in the listing 10.

```
function swap(address tokenIn, uint amountIn, address tokenOut) external {
    require(IERC20(tokenIn).transferFrom(msg.sender, address(this), amountIn), "transferFrom failed");

```

```
4     (uint reserveIn, uint reserveOut) = (reserves[tokenIn],
         reserves[tokenOut]);
5
6     uint amountOut = (amountIn * reserveOut) / (reserveIn +
         amountIn);
7
8     require(IERC20(tokenOut).transfer(msg.sender, amountOut), "
         transfer failed");
9
10    reserves[tokenIn] += amountIn;
11    reserves[tokenOut] -= amountOut;
12 }
```

Listing 10: Implementation of the remediation to the injected the SWC-104: Unchecked Call Return Value vulnerability

In the listing 10, the issue is fixed by using `require` statement in lines 2 and 8. This ensures that if the token operation fails, the whole transaction is reverted. This eliminates the SWC-104 vulnerability.

2.6 Unprotected SELFDESTRUCT Instruction (SWC-106)

The vulnerability occurs if the function containing the `selfdestruct()` or `suicide()` instruction is not properly secured, any user can call it and remove the smart contract from the blockchain, losing access to all data and funds.

The way the vulnerability was introduced into the source code is shown in the listing 11.

```
1 function destroy() external {
2     suicide(msg.sender);
3 }
```

Listing 11: Implementation of the SWC-106: Unprotected SELFDESTRUCT Instruction vulnerability in the smart contract example

In the listing 11, the function `destroy` uses `suicide(msg.sender)` without any access control. This introduces the SWC-106: Unprotected Selfdestruct vulnerability. Anyone can call this function and permanently destroy the smart contract, which would result in a total loss of funds and functionality.

The way the vulnerability could be remediated is shown in the listing 12.

```
function destroy() external {
    require(msg.sender == owner, "Not authorized");
    suicide(msg.sender);
}
```

Listing 12: Implementation of the remediation to the injected the SWC-106: Unprotected SELFDESTRUCT Instruction vulnerability

In the listing 12, a `require` statement is added in line 2 to check that `msg.sender == owner`. This ensures that only the smart contract owner can trigger self-destruction. With this access control, unauthorized users cannot call `destroy`, and the SWC-106 vulnerability is fixed.

2.7 Reentrancy (SWC-107)

The vulnerability is that the smart contract makes an external call (e.g. call, send or transfer) before updating its state. If the recipient address is another smart contract, it can immediately call the parent smart contract function again (reentrancy) before the previous execution has finished. This can lead to multiple withdrawals.

The way the vulnerability was introduced into the source code is shown in the listing 13.

```
1 function removeLiquidity(address tokenAddress, uint share) {
2     uint amount = share * reserves[tokenAddress];
3
4     IERC20(tokenAddress).transfer(msg.sender, amount);
5
6     shares[msg.sender] -= share;
7     reserves[tokenAddress] -= amount;
8 }
```

Listing 13: Implementation of the SWC-107: Reentrancy vulnerability in the smart contract example

In the listing 13, the function `removeLiquidity` first transfers tokens to `msg.sender` in line 4, and only after that updates the internal state (`shares` and `reserves`). This creates the SWC-107: Reentrancy vulnerability. A malicious smart contract could call back into `removeLiquidity` during the external transfer, before the state is updated, and withdraw more funds than allowed.

The way the vulnerability could be remediated is shown in the listing 14.

```
function removeLiquidity(address tokenAddress, uint share) {
    uint amount = share * reserves[tokenAddress];

    shares[msg.sender] -= share;
    reserves[tokenAddress] -= amount;

    IERC20(tokenAddress).transfer(msg.sender, amount);
}
```

Listing 14: Implementation of the remediation to the injected the SWC-107: Reentrancy vulnerability

In the listing 14, the state updates happen first (lines 4–5), and the external call to transfer is moved to the end (line 7). This ensures that even if a reentrant call is attempted, the internal state is already updated and cannot be exploited. This removes the SWC-107 vulnerability.

2.8 State Variable Default Visibility (SWC-108)

In Solidity, if the visibility of a state variable is not explicitly declared, it is considered internal by default. Failure to explicitly declare it can lead to confusion about data accessibility and make source code auditing difficult. Best practices recommend always explicitly specifying the visibility of each state variable (`public`, `internal`, `private`).

The way the vulnerability was introduced into the source code is shown in the listing 15.

```
uint unused;
```

Listing 15: Implementation of the SWC-108: State Variable Default Visibility vulnerability in the smart contract example

In the listing 15, the state variable `unused` is declared without a visibility specifier. In Solidity, the default visibility for state variables is internal, but leaving it implicit can cause confusion and reduce code clarity. This is classified as SWC-108: State Variable Default Visibility. An unclear visibility declaration makes it harder to audit the smart contract and may lead to unintended assumptions about access.

The way the vulnerability could be remediated is shown in the listing 16.

```
uint private unused;
```

Listing 16: Implementation of the remediation to the injected the SWC-108: State Variable Default Visibility vulnerability

In the listing 16, the variable is explicitly marked as private. This makes the intended visibility clear and removes ambiguity. By explicitly declaring visibility, the source code is more comprehensible and the SWC-108 issue is resolved.

2.9 Uninitialized Storage Pointer (SWC-109)

Failure to initialise pointers in memory can lead to them accidentally pointing to a common or incorrect location in memory, which can result in unpredictable smart contract performance.

The way the vulnerability was introduced into the source code is shown in the listing 17.

```
1  function addLiquidity(address tokenAddress, uint amount) external
       {
2      IERC20(tokenAddress).transferFrom(msg.sender, address(this),
           amount);
3
4      LiquidityProvider storage lp;
5      lp.amount = amount;
6      lp.lastAdded = block.timestamp;
7
8      shares[msg.sender] =+ amount;
9      reserves[tokenAddress] += amount;
10 }
```

Listing 17: Implementation of the SWC-109: Uninitialized Storage Pointer vulnerability in the smart contract example

In the listing 17, the variable `lp` is declared as `LiquidityProvider storage lp;` in line 4 but never initialized to a valid storage reference. This creates the SWC-109: Uninitialized Storage Pointer vulnerability. Uninitialized storage pointers may reference arbitrary storage slots, so writing to them can overwrite critical data in the smart contract, leading to corruption of state variables.

The way the vulnerability could be remediated is shown in the listing 18.

```
function addLiquidity(address tokenAddress, uint amount) external {
    IERC20(tokenAddress).transferFrom(msg.sender, address(this), amount);

    LiquidityProvider storage lp = providers[msg.sender];
    lp.amount = amount;
    lp.lastAdded = block.timestamp;

    shares[msg.sender] =+ amount;
    reserves[tokenAddress] += amount;
}
```

Listing 18: Implementation of the remediation to the injected the SWC-109: Uninitialized Storage Pointer vulnerability

In the listing 18, the issue is fixed by explicitly initializing `lp` with a reference to `providers[msg.sender]` in line 4. This ensures that `lp` correctly points to the storage location assigned to the specific liquidity provider. As a result, data is stored in the intended mapping entry and the SWC-109 vulnerability is removed.

2.10 Assert Violation (SWC-110)

The `assert()` function in Solidity is designed to check for internal logical errors. Its use in validation situations can lead to suboptimal error management, as it consumes all available gas if it fails. It is recommended to use `require()` function to check input conditions and `assert()` function only in cases where conditions should be logically impossible to break.

The way the vulnerability was introduced into the source code is shown in the listing 19.

```
function getAddressAmount(address user) external view returns (uint) {
    assert(false);

    providers[user].amount;
}
```

Listing 19: Implementation of the SWC-110: Assert Violation vulnerability in the smart contract example

In the listing 19, the function `getAddressAmount` uses `assert(false)` in line 2. This creates the SWC-110: Assert Violation vulnerability. The `assert(false)` statement makes the function unusable, since it will always revert.

The way the vulnerability could be remediated is shown in the listing 20.

```
function getAddressAmount(address user) external view returns (uint) {
    providers[user].amount;
}
```

Listing 20: Implementation of the remediation to the injected the SWC-110: Assert Violation vulnerability

In the listing 20, the `assert(false)` line is removed, and the function directly returns the user's amount. This ensures normal execution and proper gas usage. By removing the incorrect assert, the SWC-110 vulnerability is resolved.

2.11 Use of Deprecated Solidity Functions (SWC-111)

Some functions and constructs in Solidity are marked as deprecated over time, such as `sha3`, `throw`, and `suicide` functions. Their continued use can lead to unpredictable errors and compatibility problems in the future. Instead, it is recommended to use current equivalents, such as `keccak256` function instead of `sha3` function, `revert` instruction instead of `throw` instruction, and `selfdestruct` instead of `suicide`.

The `keccak256` function replaced `sha3` to avoid ambiguity in naming and to explicitly refer to the standardized Keccak-256 hashing algorithm. This ensures clarity and aligns Solidity with the Ethereum Yellow Paper specification.

The `revert` instruction was introduced as a clearer and safer replacement for `throw`. While `throw` reverted the transaction and consumed all remaining gas, `revert` allows developers to revert execution while preserving unused gas and additionally can provide a descriptive error message. This improves both developer experience and smart contract usability.

The `selfdestruct` instruction replaced `suicide` to make the semantics of the function more transparent. Its behavior remains the same (removing code and storage from the blockchain and sending remaining ether to a designated address), but the new naming avoids negative connotations and better reflects the actual intent of the operation.

The way the vulnerability was introduced into the source code is shown in the listing 21.

```
function destroy() external {
    suicide(msg.sender);
}
```

Listing 21: Implementation of the SWC-111: Use of Deprecated Solidity Functions vulnerability in the smart contract example

In the listing 21, the `suicide` instruction was used in line 2. This causes SWC-111: Use of Deprecated Solidity Functions.

The way the vulnerability could be remediated is shown in the listing 22.

```
function destroy() external {
    selfdestruct(msg.sender);
}
```

Listing 22: Implementation of the remediation to the injected the SWC-111: Use of Deprecated Solidity Functions vulnerability

In the listing 22 `suicide` statement is replaced by `selfdestruct` statement. This ensures more predictable and compatible code. By replacing statements, the SWC-111 vulnerability is resolved.

2.12 Delegatecall to Untrusted Callee (SWC-112)

Function `delegatecall()` allows source code of another smart contract to be executed in the context of the calling smart contract. If the address is user-controlled or not checked, malicious source code may be executed.

The way the vulnerability was introduced into the source code is shown in the listing 23.

```
function delegateTo(address callee, bytes data) external {
    require(callee.callcode(data));
}
```

Listing 23: Implementation of the SWC-112: Delegatecall to Untrusted Callee vulnerability in the smart contract example

In the listing 23, the function `delegateTo` allows anyone to call `callee.callcode(data)` without restriction. This creates the SWC-112: Delegatecall to Untrusted Callee vulnerability. An attacker could pass a malicious smart contract as `callee` and execute arbitrary

code in the context of the vulnerable smart contract, potentially stealing funds.

The way the vulnerability could be remediated is shown in the listing 24.

```
function delegateTo(address callee, bytes data) external {
    require(msg.sender == owner, "Not authorized");
    require(callee.callcode(data));
}
```

Listing 24: Implementation of the remediation to the injected the SWC-112: Delegatecall to Untrusted Callee vulnerability

In the listing 24, a `require` statement is added in line 2 to restrict access. This ensures that only the smart contract owner can perform delegate calls. By limiting access, untrusted users cannot execute arbitrary code, which mitigates the SWC-112 vulnerability.

2.13 DoS with Failed Call (SWC-113)

This type of vulnerability occurs when a smart contract performs an operation, which execution time increases non-linearly depending on the data (e.g., iterating over a dynamic array). This can lead to a gas overrun and consequently block the functionality of the smart contract, i.e. might be used to perform a Denial of Service attack on the application.

The way the vulnerability was introduced into the source code is shown in the listing 25.

```
function triggerFail() public {
    require(address(this).call(bytes4(keccak256("nonExistent()"))
        ));
}
```

Listing 25: Implementation of the SWC-113: DoS with Failed Call vulnerability in the smart contract example

In the listing 25, the function `triggerFail` uses a low-level call via `address(this).call(...)` without properly checking the return value in line 2. This creates the SWC-113: DoS with Failed Call vulnerability. If the called function does not exist or fails, the original require may not handle the failure correctly, which can lead to unexpected reverts or denial of service.

The way the vulnerability could be remediated is shown in the listing 26.

```
fallback() external {
}

function triggerFail() public {
    (bool success, ) = address(this).call(bytes4(keccak256("
        nonExistent()"))));
    require(success, "Low-level call failed");
}
```

Listing 26: Implementation of the remediation to the injected the SWC-113: DoS with Failed Call vulnerability

In the listing 26, the low-level call return value is captured with `(bool success, )` and explicitly checked using `require(success, "Low-level call failed")`. This ensures that if the call fails, the transaction reverts with a clear error message. By properly handling the result of the call, the SWC-113 vulnerability is mitigated.

2.14 Authorization through tx.origin (SWC-115)

Instruction `tx.origin` returns the original sender of the transaction, which can be exploited by malicious smart contracts mediating the transaction. Instead, instruction `msg.sender` should be used, which indicates the direct sender of the current call.

The way the vulnerability was introduced into the source code is shown in the listing 27.

```
1 function onlyOwner() public {
2     require(tx.origin == owner);
3 }
```

Listing 27: Implementation of the SWC-115: Authorization through tx.origin vulnerability in the smart contract example

In the listing 27, the function `onlyOwner` uses `require(tx.origin == owner)` in line 2. This introduces the SWC-115: tx.origin Authentication vulnerability. Using `tx.origin` for authentication is unsafe because a malicious smart contract can trick the owner into initiating a transaction, allowing the attacker to execute `onlyOwner` indirectly and gain unauthorized access.

The way the vulnerability could be remediated is shown in the listing 28.

```
function onlyOwner() public {
    require(msg.sender == owner, "Not owner");
}
```

Listing 28: Implementation of the remediation to the injected the SWC-115: Authorization through tx.origin vulnerability

In the listing 28, the check is replaced with `require(msg.sender == owner, "Not owner")` in line 2. Using `msg.sender` ensures that only the direct caller (the owner) can execute the function, preventing attacks through intermediary smart contracts. This change removes the SWC-115 vulnerability.

2.15 Block values as a proxy for time (SWC-116)

Using block fields, such as block.timestamp or block.number, as a time source can be dangerous, as they can be slightly modified by miners. If the smart contract logic depends on time accuracy, this can be used for manipulation.

The way the vulnerability was introduced into the source code is shown in the listing 29.

```
function getRandomNumber() public view returns (uint) {
    return uint(blockhash(block.number - 1)) % 100 + block.
        timestamp;
}
```

Listing 29: Implementation of the SWC-116: Block values as a proxy for time vulnerability in the smart contract example

In the listing 29, the function `getRandomNumber` uses `block.timestamp` and `blockhash` to generate a number. This introduces the SWC-116: Block Values as a Proxy for Time vulnerability. Using `block.timestamp` as a source of timing or sequencing can be manipulated by miners, which may affect logic depending on time, such as time-based restrictions or ordering of actions.

The way the vulnerability could be remediated is shown in the listing 30.

```
uint private nonce;

function getRandomNumber(address user) public returns (uint) {
    nonce++;

    return uint(keccak256(abi.encodePacked(user, nonce))) % 100;
}
```

Listing 30: Implementation of the remediation to the injected the SWC-116: Block values as a proxy for time vulnerability

In the listing 30, the function replaces reliance on `block.timestamp` with an internal `nonce` that increments on each call. By using `nonce` combined with the user address and `keccak256`, the smart contract no longer depends on blockchain timestamps. This prevents miners from influencing smart contract behavior based on timestamp manipulation and resolves the SWC-116 vulnerability.

2.16 Signature Malleability (SWC-117)

The vulnerability relates to the ability to modify a digital signature without knowing the private key. In Ethereum, due to the way ECDSA signatures work, it is possible to create more than one valid signature for the same message. If a smart contract does not verify signatures in an appropriate manner (e.g., using ecrecover without additional security), this can lead to serious consequences, such as re-executing the same operation.

The way the vulnerability was introduced into the source code is shown in the listing 31.

```
function verify(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
   public pure returns (bool) {
    ecrecover(hash, v, r, s); // No s value validation
    return true;
}
```

Listing 31: Implementation of the SWC-117: Signature Malleability vulnerability in the smart contract example

In the listing 31, the function `verify` calls `ecrecover` function without validating the `s` and `v` values. This introduces the SWC-117: Signature Malleability vulnerability. Without checking `s` and `v` values, an attacker can submit an alternative signature that is still valid but has a different encoding, potentially allowing replay attacks or bypassing

signature-based checks.

The way the vulnerability could be remediated is shown in the listing 32.

```
function verify(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
    public pure returns (bool) {
    bytes32 SECP256K1N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFE
        FFEBAAEDCE6AF48A03BBFD25E8CD0364141;
    if (uint256(s) > uint256(SECP256K1N) / 2) {
        return false;
    }

    if (v != 27 && v != 28) {
        return false;
    }

    address signer = ecrecover(hash, v, r, s);
    return signer != address(0);
}
```

Listing 32: Implementation of the remediation to the injected the SWC-117: Signature Malleability vulnerability

In the listing 32, the function adds proper validation:

- `s` value is checked to be at most `SECP256K1N / 2` to enforce a canonical signature,
- `v` value is checked to be either 27 or 28, to ensure the signature uses a valid Ethereum format and can be correctly verified by `ecrecover` function,
- the recovered address is verified to be non-zero.

These checks ensure that only valid signatures are accepted, preventing signature malleability and removing the SWC-117 vulnerability.

2.17 Incorrect Constructor Name (SWC-118)

In older versions of Solidity (before version 0.4.22), the constructor was identified by the function name corresponding to the smart contract name. A typo or file name change could lead to the constructor not being recognized as such, and becoming a public function. This could have allowed an attacker to take unauthorized actions, such as taking ownership of the smart contract.

The way the vulnerability was introduced into the source code is shown in the listing 33.

```
function SimpleDex() public {
    owner = msg.sender;
}
```

Listing 33: Implementation of the SWC-118: Incorrect Constructor Name vulnerability in the smart contract example

In the listing 33, the smart contract uses a function named `SimpleDex` as a constructor. This is old Solidity syntax and is deprecated. If the smart contract name changes, this function becomes a normal public function, and anyone could call it to take ownership of the smart contract.

The way the vulnerability could be remediated is shown in the listing 34.

```
constructor () public {
    owner = msg.sender;
}
```

Listing 34: Implementation of the remediation to the injected the SWC-118: Incorrect Constructor Name vulnerability

In the listing 34, the smart contract uses `constructor()` instead. This runs only once when the smart contract is deployed and cannot be called later. Using `constructor()` fixes the SWC-118 issue.

2.18 Shadowing State Variables (SWC-119)

Variable shadowing refers to a situation where a local variable or one defined within a function obscures a global or inherited variable with the same name. This can result in unintended behavior and logical errors.

The way the vulnerability was introduced into the source code is shown in the listing 35.

```
mapping(address => uint) public shares;

function shadowVariable() public {
    uint shares = 123;
}
```

Listing 35: Implementation of the SWC-119: Shadowing State Variables vulnerability in the smart contract example

In the listing 35, the function `shadowVariable` declares a local variable `shares` in line 4, that has the same name as the state variable `shares` declared in line 1. This creates the SWC-119: Shadowing State Variables vulnerability. The local variable hides the state variable, so any updates affect only the local variable and not the smart contract's storage, which can cause unexpected behavior or bugs.

The way the vulnerability could be remediated is shown in the listing 36.

```
mapping(address => uint) public shares;

function shadowVariable() public {
    uint newShares = 123;
}
```

Listing 36: Implementation of the remediation to the injected the SWC-119: Shadowing State Variables vulnerability

In the listing 36, the local variable is renamed to `newShares`. This removes the name conflict, so the state variable shares is no longer shadowed. This fixes the SWC-119 issue.

2.19 Weak Sources of Randomness from Chain Attributes (SWC-120)

Values such as `block.timestamp`, `block.difficulty`, or `block.hash` are not random enough and can be predictable. Using them to generate randomness in games, lotteries or elections is vulnerable to manipulation by miners or attackers.

The way the vulnerability was introduced into the source code is shown in the listing 37.

```
function getRandomNumber() public view returns (uint) {
    return uint(blockhash(block.number - 1)) % 100 + block.
        timestamp;
}
```

Listing 37: Implementation of the SWC-120: Weak Sources of Randomness from Chain Attributes vulnerability in the smart contract example

This is how the vulnerability could be remediated:

- Using external sources of randomness via oracles, and cryptographically checking the outcome of the oracle on-chain. e.g. Chainlink VRF. This approach mitigates SWC-120 because the randomness is generated off-chain in a cryptographically secure way

and then verified on-chain. As a result, it is not possible for miners or smart contract participants to manipulate predictable block attributes to influence the outcome.

- Using commitment scheme, e.g. RANDAO. A commitment scheme addresses SWC-120 by distributing the process of randomness generation among multiple participants. Since each party first commits to a value and only later reveals it, a single actor cannot unilaterally influence the result by exploiting predictable chain attributes such as timestamps or block hashes.

- Using external sources of randomness via oracles, e.g. Oraclize. Although, this approach introduces a trust assumption, it mitigates SWC-120 because randomness does not rely on easily manipulable chain attributes. Instead, it is provided by an external oracle, reducing the ability of miners or smart contract users to predict or bias the random outcome. Using multiple independent oracles further decreases the risk of relying on weak, on-chain randomness.

2.20 Missing Protection against Signature Replay Attacks (SWC-121)

The vulnerability lies in the lack of safeguards against reuse (replay) of a signed message. If a smart contract accepts signatures and does not verify the uniqueness or timeliness of the message (e.g., by nonce or timestamp), an attacker can use the same signature multiple times, maliciously performing an identical operation.

The way the vulnerability was introduced into the source code is shown in the listing 38.

```
function submitSigned(bytes32 hash, uint8 v, bytes32 r, bytes32 s
    ) external {
    address signer = ecrecover(hash, v, r, s);
    require(signer != address(0));
    usedSignatures[hash] = true;
}
```

Listing 38: Implementation of the SWC-121: Missing Protection against Signature Replay Attacks vulnerability in the smart contract example

In the listing 38, the function `submitSigned` stores each `hash` in `usedSignatures` without linking it to a `nonce`. This creates the SWC-121: Missing Protection against Signature Replay Attacks vulnerability. An attacker could reuse a previously submitted signature to execute the same action multiple times, bypassing the intended restrictions.

The way the vulnerability could be remediated is shown in the listing 39.

```
mapping(address => uint256) public nonces;
mapping(bytes32 => bool) public usedSignatures;

function submitSigned(
    bytes32 hash,
    uint8 v,
    bytes32 r,
    bytes32 s
) external {
    address signer = ecrecover(hash, v, r, s);
    require(signer != address(0), "Invalid signature");

    bytes32 expectedHash = keccak256(abi.encodePacked(signer,
        nonces[signer]));
    require(hash == expectedHash, "Incorrect hash or nonce");

    require(!usedSignatures[hash], "Signature already used");
    usedSignatures[hash] = true;

    nonces[signer]++;
}
```

Listing 39: Implementation of the remediation to the injected the SWC-121: Missing Protection against Signature Replay Attacks vulnerability

In the listing 39, the function is updated to include a per signer `nonce` stored in the `nonces` mapping. Each submitted `hash` must match `keccak256(abi.encodePacked(signer, nonces[signer]))` and is checked against `usedSignatures`. After a successful submission, `nonces[signer]` is incremented. This ensures that each signature can only be used once, preventing replay attacks and resolving the SWC-121 vulnerability.

2.21 Lack of Proper Signature Verification (SWC-122)

This vulnerability occurs when a smart contract incorrectly verifies signatures - for example, it fails to validate the signer's address, fails to validate the data format, or fails to consider the scope of the data covered by the signature. This can allow fraudulent authorizations and actions on behalf of another user without the user's knowledge.

The way the vulnerability was introduced into the source code is shown in the listing 40.

```
function verify(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
   public pure returns (bool) {
    ecrecover(hash, v, r, s);
    return true;
}
```

Listing 40: Implementation of the SWC-122: Lack of Proper Signature Verification vulnerability in the smart contract example

In the listing 40, the function `verify` calls `ecrecover` but does not validate the `s` and `v` values. This creates the SWC-122: Improper Signature Verification vulnerability. Without these checks, an attacker can submit a malleable signature that still passes `ecrecover`, potentially bypassing signature-based authentication or validation.

The way the vulnerability could be remediated is shown in the listing 41.

```
function verify(bytes32 hash, uint8 v, bytes32 r, bytes32 s)
   public pure returns (bool) {
    bytes32 SECP256K1N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
       FFEBAAEDCE6AF48A03BBFD25E8CD0364141;
    if (uint256(s) > uint256(SECP256K1N) / 2) {
        return false;
    }

    if (v != 27 && v != 28) {
        return false;
    }

    address signer = ecrecover(hash, v, r, s);
    return signer != address(0);
}
```

Listing 41: Implementation of the remediation to the injected the SWC-122: Lack of Proper Signature Verification vulnerability

In the listing 41, the function adds explicit validation:

- `s` value is checked to be at most `SECP256K1N / 2` to enforce a canonical signature,
- `v` value is checked to be either 27 or 28, to ensure the signature uses a valid Ethereum format and can be correctly verified by `ecrecover` function,
- the recovered address is verified to be non-zero.

The function then returns the valid signer address. By adding these checks, only proper, non-malleable signatures are accepted, resolving the SWC-122 vulnerability.

This remediation is the same as the one presented above for SWC-117, as it simultaneously addresses both vulnerabilities.

2.22 DoS With Block Gas Limit (SWC-128)

This vulnerability involves creating functions that consume too much gas in a single transaction. If an operation, such as iterating over a large array, exceeds the block's gas limit, the function fails to execute. This can lead to a situation where certain operations become impossible, resulting in a Denial of Service (DoS).

The way the vulnerability was introduced into the source code is shown in the listing 42.

```
function loopUsers() public {
    for (uint i = 0; i < users.length; i++) {
        address _user = users[i];
        shares[_user] += 1;
    }
}
```

Listing 42: Implementation of the SWC-128: DoS With Block Gas Limit vulnerability in the smart contract example

In the listing 42, the function `loopUsers` iterates over the entire `users` array in a single transaction. This creates the SWC-128: DoS with Block Gas Limit vulnerability. If the `users` array grows too large, the transaction could exceed the block gas limit and fail, effectively causing a denial of service.

The way the vulnerability could be remediated is shown in the listing 43.

```
function loopUsers(uint start, uint end) public {
    require(end <= users.length && start < end, "Invalid range");

    for (uint i = start; i < end; i++) {
        address _user = users[i];
        shares[_user] += 1;
    }
}
```

Listing 43: Implementation of the remediation to the injected the SWC-128: DoS With Block Gas Limit vulnerability

In the listing 43, the function is updated to accept `start` and `end` indices and checks that `start < end` and `end <= users.length`. This allows processing the array in smaller batches, preventing gas limit issues and mitigating the SWC-128 vulnerability.

2.23 Typographical Error (SWC-129)

It refers to typos in code, such as misspelled identifiers for variables, functions or libraries. Although seemingly minor, such errors can cause unexpected smart contract performance, compilation errors or the use of inappropriate resources, negatively affecting security and functionality.

The way the vulnerability was introduced into the source code is shown in the listing 44.

```
function addLiquidity(address tokenAddress, uint amount) external {
    IERC20(tokenAddress).transferFrom(msg.sender, address(this), amount);

    LiquidityProvider storage lp;
    lp.amount = amount;
    lp.lastAdded = block.timestamp;

    shares[msg.sender] =+ amount;
    reserves[tokenAddress] += amount;
}
```

Listing 44: Implementation of the SWC-129: Typographical Error vulnerability in the smart contract example

In the listing 44, the function `addLiquidity` uses the statement `shares[msg.sender] =+ amount` in line 7. This is a typo: instead of incrementing the existing value, it assigns a positive value of `amount`, effectively overwriting the previous balance. This creates the SWC-129: Typographical error vulnerability, which can lead to incorrect accounting of user shares.

The way the vulnerability could be remediated is shown in the listing 45.

```
function addLiquidity(address tokenAddress, uint amount) external {
    IERC20(tokenAddress).transferFrom(msg.sender, address(this), amount);

```

```
    LiquidityProvider storage lp;
    lp.amount = amount;
    lp.lastAdded = block.timestamp;

    shares[msg.sender] += amount;
    reserves[tokenAddress] += amount;
}
```

Listing 45: Implementation of the remediation to the injected the SWC-129: Typographical Error vulnerability

In the listing 45, the operator is corrected to `shares[msg.sender] += amount`, which properly adds `amount` to the existing balance. This ensures correct accounting and removes the SWC-129 vulnerability.

2.24 Presence of unused variables (SWC-131)

Variables that are declared but not used increase the complexity of the smart contract and can introduce ambiguity. Although they do not pose an immediate security risk, they can be indicative of logical errors or unconsidered code.

The way the vulnerability was introduced into the source code is shown in the listing 46.

```
uint unused;
```

Listing 46: Implementation of the SWC-131: Presence of unused variables vulnerability in the smart contract example

The presence of unused variables might be remediated by using such variables in the source code or simply removing their declaration.

2.25 Unexpected Ether balance (SWC-132)

This vulnerability occurs when a smart contract assumes that its balance (Ether amount) comes only from certain functions (e.g., `payable`). In reality, a smart contract's balance can be increased externally, for example, by using the self-destruct function in another smart contract. If the source code does not take such situations into account, the smart contract logic can malfunction.

The way the vulnerability was introduced into the source code is shown in the listing 47.

```
function () public payable {}
```

Listing 47: Implementation of the SWC-132: Unexpected Ether balance vulnerability in the smart contract example

In the listing 47, the smart contract defines a fallback function `function() public payable` that accepts Ether without any restriction. This creates the SWC-132: Unexpected Ether balance vulnerability. Users might accidentally send Ether to the smart contract, which could be locked there forever since there is no withdrawal mechanism.

The way the vulnerability could be remediated is shown in the listing 48.

```
function () public payable {
    revert("Contract does not accept Ether");
}
```

Listing 48: Implementation of the remediation to the injected the SWC-132: Unexpected Ether balance vulnerability

In the listing 48, the fallback function is updated to immediately `revert("Contract does not accept Ether")`. This prevents the smart contract from unintentionally receiving Ether, mitigating the SWC-132 vulnerability.

2.26 Hash Collisions With Multiple Variable Length Arguments (SWC-133)

It occurs when `abi.encodePacked()` function is used with multiple variable-length arguments, such as strings, bytes. Hash collisions can occur, allowing attackers to manipulate the input data to produce the same hash function result. This can lead to unauthorized access or data validation errors.

The way the vulnerability was introduced into the source code is shown in the listing 49.

```
function collisionExample(string memory a, uint b) public pure
   returns (bytes32) {
    return keccak256(abi.encodePacked(a, b));
}
```

Listing 49: Implementation of the SWC-133: Hash Collisions With Multiple Variable Length Arguments vulnerability in the smart contract example

In the listing 49, the function `collisionExample` uses `keccak256(abi.encodePacked(a, b))` to generate a hash. This creates the SWC-133: Hash Collisions With Multiple Variable Length Arguments vulnerability. When two variables of dynamic types (like `string` and `uint`) are concatenated with `abi.encodePacked`, different input pairs can produce the same hash, potentially leading to collisions and unexpected behavior.

The way the vulnerability could be remediated is shown in the listing 50.

```
function collisionExample(string memory a, uint b) public pure
    returns (bytes32) {
    return keccak256(abi.encode(a, b));
}
```

Listing 50: Implementation of the remediation to the injected the SWC-133: Hash Collisions With Multiple Variable Length Arguments vulnerability

In the listing 50, the function uses `keccak256(abi.encode(a, b))` instead. Unlike `encodePacked`, `abi.encode` preserves type information and padding, preventing collisions for different input pairs. This removes the SWC-133 vulnerability.

2.27 Message call with hardcoded gas amount (SWC-134)

This vulnerability involves the use of a predetermined (hardcoded) gas value when calling the call function. This type of construction is dangerous because changes in the cost of EVM operations can cause the smart contract to stop working properly (e.g., the recipient does not receive enough gas). This can also be used for a denial of service (DoS) attack.

The way the vulnerability was introduced into the source code is shown in the listing 51.

```
function lowGasCall(address target, bytes calldata data) external
    {
    require(target.call.gas(10000)(data));
}
```

Listing 51: Implementation of the SWC-134: Message call with hardcoded gas amount vulnerability in the smart contract example

In the listing 51, the function `lowGasCall` uses a low-level call with a fixed gas. This creates the SWC-134: Message call with hardcoded gas amount vulnerability. If the called smart contract requires more than 10,000 gas to execute, the call will fail unexpectedly, leading to denial of service or failed logic.

The way the vulnerability could be remediated is shown in the listing 52.

```
function lowGasCall(address target, bytes calldata data) external {
    (bool success, ) = target.call(data);
    require(success, "Low-level call failed");
}
```

Listing 52: Implementation of the remediation to the injected the SWC-134: Message call with hardcoded gas amount vulnerability

In the listing 52, fixed gas limit is removed, allowing the called smart contract to execute with the gas available in the transaction. The explicit check of `success` ensures proper error handling, mitigating the SWC-134 vulnerability.

2.28 Code With No Effects (SWC-135)

This type of vulnerability applies to source code fragments that have no effect on the state of the smart contract or its outcome. For example, reading the value of a variable without using it, or assigning a value that is not used anywhere. Such code increases complexity and can hide logical errors.

The way the vulnerability was introduced into the source code is shown in the listing 53.

```
function getAddressAmount(address user) external view returns (uint) {
    assert(false);

    providers[user].amount;
}
```

Listing 53: Implementation of the SWC-135: Code With No Effects vulnerability in the smart contract example

This is how the vulnerability could be remediated: Either remove unnecessary statements or make them useful.

Remediation example is shown in the listing 54

```
function getAddressAmount(address user) external view returns (
    uint) {
     assert(false);

     return providers[user].amount;
}
```

Listing 54: Implementation of the remediation to the injected the SWC-135: Code With No Effects vulnerability

2.29 Unencrypted Private Data On-Chain (SWC-136)

The data stored in the smart contract is publicly available, even if marked as private. Anyone can read them from the blockchain. Sensitive information, such as keys, passwords or personal data, should never be stored directly in a smart contract without proper encryption.

The way the vulnerability was introduced into the source code is shown in the listing 55.

```
string public secretAdminPassword = "admin1234";
```

Listing 55: Implementation of the SWC-136: Unencrypted Private Data On-Chain vulnerability in the smart contract example

In the listing 55, the smart contract stores the admin password as a public string. This creates the SWC-136: Unencrypted Private Data On-Chain vulnerability. Anyone can read the password directly from the blockchain, which allows unauthorized users to gain access.

This is how the vulnerability could be remediated: Either remove sensitive data or store it hashed only.

Remediation example is shown in the listing 56.

```
bytes32 private secretAdminPasswordHash;

constructor(bytes32 _hash) public {
    secretAdminPasswordHash = _hash;
}
```

Listing 56: Implementation of the remediation to the injected the SWC-136: Unencrypted Private Data On-Chain vulnerability

In the listing 56, the smart contract stores only a private hash of the password, which is set in the constructor. By keeping the secret as a hashed value and private, it is no longer exposed on-chain, mitigating the SWC-136 vulnerability.